

Improving the KMP Algorithm by Using Properties of Fibonacci String

Yi-Kung Shieh* and R. C. T. Lee

Department of Computer Science

National Tsing Hua University

d9762814@oz.nthu.edu.tw and rctlee@ncnu.edu.tw

Abstract

In this paper, we explain a special string, Fibonacci string. When Fibonacci strings exist in prefixes of inputted pattern, it may cause a delayed situation in searching text by using the KMP algorithm. We discuss the conditions of delayed situation. We use the preprocessing table of the KMP algorithm to find Fibonacci strings which exist in prefixes of pattern. After we know the reasons which cause the delayed situation, we use a simple concept to avoid such situation.

1 Introduction

The KMP algorithm is to solve the exact string matching problem [1]. The definition of exact string matching problem is: Given a text $T=t_1t_2\dots t_n$ and a pattern $P=p_1p_2\dots p_m$, find all occurrences of the pattern in the text. In this paper, we discuss if there exists a prefix of inputted pattern which consists of recursive palindromes, it may cause a delayed situation in searching phase by using the KMP algorithm. Fibonacci string contains such prefixes [3]. In the section 3, we introduce the conceptions of palindrome and Fibonacci string. After observing the property of Fibonacci string, we improve the delayed situation by using *failure* table of the KMP algorithm. We utilize the *failure* table to construct a new table whose size is equal to *failure* table's. The new table records the above prefixes. Hence, if the delayed situation occurs, we can use the new table to avoid it.

2 The KMP algorithm and its delayed situation

The algorithm contains two phases, preprocessing and searching phase. In preprocessing phase, it constructs a *failure* table with size m . For location i ($1 \leq i \leq m$), if j is the largest such that the prefix $P(1, j-1)$ ($P(1, j-1)$ means $p_1p_2\dots p_{j-1}$) is a suffix of $P(1, i-1)$ and $p_i \neq p_j$, $failure[i] = j$. If no such j exists but $p_i \neq p_1$, $failure[i] = 1$. If no such j exists and $p_i = p_1$, $failure[i] = 0$. For example, $P = babbabab$. Its *failure* table is shown as Table 1.

In $i = 6$, we cannot find any j such that $P(1, j-1)$ is a prefix of $P(1, i-1) = P(1, 6-1) = P(1, 5)$, and $p_6 = p_1$. Therefore, $failure[6] = 0$. In $i = 10$, we also cannot find such j but $p_i = p_{10} \neq p_1$. So, $failure[10] = 1$. In $i = 12$, we find that two prefixes $P(1, 2-1) = P(1, 1) = b$ and $P(1, 7-1) = P(1, 6) = babbab$ both are suffixes of $P(1, i-1) = P(1, 12-1) = P(1, 11) = babbababbab$. $p_{12} = b \neq a = p_2$ and $p_{12} = b \neq a = p_7$. We only want the largest one. Hence, $failure[12] = 7$.

In searching phase, it opens a window with length m in the leftmost of text initially. Compare the window with pattern from left to right. If first mismatch occurs in location i of pattern, we shift the window right. When we want to shift the window, we use the *failure* table which had been constructed in preprocessing phase to determine the shifting steps. And we must remember the value of $failure[i]$. We use an integer e to record it. For above example, if the first mismatch occurs in position 7 of pattern, we use the *failure* table to shift the window $i - failure[7] = 7 - 4 = 3$ steps and $e = failure[7] = 4$. After shifting the window, we continue to compare the window with pattern from e to m . Repeat this method until the first position of the window is large than $n - m$.

Now, we roughly explain the delayed situation of the KMP algorithm by an example. In fact, the above example, $P = babbababbabba$, may cause the

*To whom correspondence should be addressed:
d9762814@oz.nthu.edu.tw

Table 1: An example of *failure* table of the KMP algorithm

i	1	2	3	4	5	6	7	8	9	10	11	12	13
Pattern	b	a	b	b	a	b	a	b	b	a	b	b	a
$failure[i]$	0	1	0	2	1	0	4	0	2	1	0	7	1

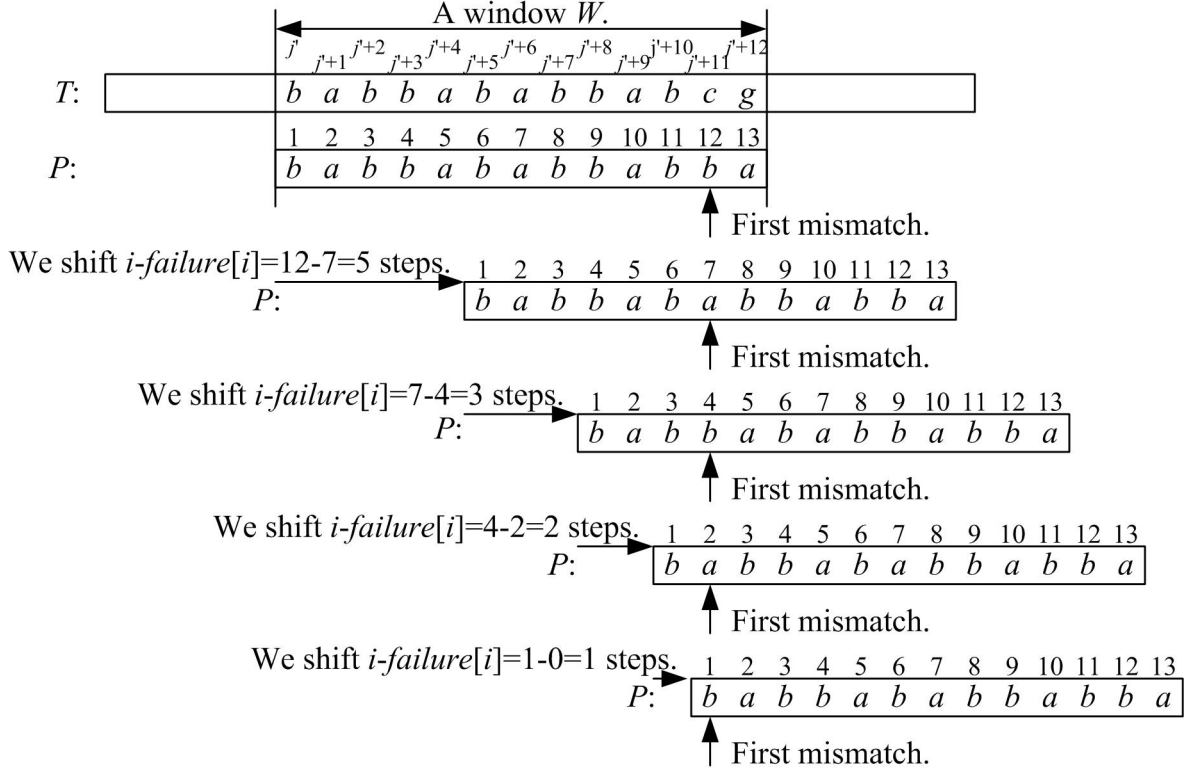


Figure 1: The delayed situation of the KMP algorithm

delayed situation. If the window whose first location is j' compare with pattern and first mismatch occurs in location $j' + 11$ of text, it causes the delayed situation as shown in Figure 1.

We can observe that the KMP algorithm sticks in the location $j' + 11$ of text. The character of location $j' + 11$ must be compared 5 times. We will explain the reason which causes the delayed situation in next section.

3 Palindrome and Fibonacci String

In palindrome, there are two types, odd palindrome and even palindrome. w is a string. w^r is the reversal of w . The form of even palindromes are ww^r . Odd palindromes are similar regularities (strings of the form $wsww^r$, where s is a character). For example, $w = ba$. We know that $w^r = ab$. Hence, $ww^r = baab$ is an even palindrome. When

$s = c$, $wsww^r = bacab$ is an odd palindromes.

We had mentioned if recursive palindromes exist in prefixes of pattern, the KMP algorithm may cause delayed situation such like Figure 1. We explain the palindromes exist in Fibonacci string. Fibonacci string comes from Fibonacci number. The definition of Fibonacci number is as follows

$$f_1 = 1. f_2 = 1. f_k = f_{k-1} + f_{k-2} \quad \text{for } k \leq 3. \quad (1)$$

By rule (1), we have $f_3 = f_{3-1} + f_{3-2} = f_2 + f_1 = 1 + 1 = 2$, $f_4 = f_{4-1} + f_{4-2} = f_3 + f_2 = 2 + 1 = 3$, $f_5 = f_{5-1} + f_{5-2} = f_4 + f_3 = 3 + 2 = 5$, $f_6 = f_{6-1} + f_{6-2} = f_5 + f_4 = 5 + 3 = 8$, etc. The Fibonacci string uses the same method. The definition of Fibonacci string is as follows

$$F_1 = x. F_2 = y. F_k = F_{k-1}F_{k-2} \quad \text{for } k \leq 3. \quad (2)$$

x and y are two different characters. When $x = a$ and $y = b$, we obtain that $F_3 = F_{3-1}F_{3-2} =$

$F_2F_1 = ba$, $F_4 = F_{4-1}F_{4-2} = F_3F_2 = bab$, $F_5 = F_{5-1}F_{5-2} = F_4F_3 = babba$, $F_6 = F_{6-1}F_{6-2} = F_5F_4 = babbabab$, etc. The Fibonacci string has special properties.

Theorem 1. For $k \geq 3$, $F_k = \alpha_k \beta_k$ where β_k is the last two characters of F_k and $\beta_k = yx$ if k is odd and $\beta_k = xy$ if k is even.

Proof. ($F_1 = x = a$, $F_2 = y = b$.)

(1) When $k=3$, $F_k = F_3 = F_2F_1 = ba$. k which is odd satisfies $\beta_k = \beta_3 = ba = yx$. When $k=4$, $F_k = F_4 = F_3F_2 = bab$. k which is even satisfies $\beta_k = \beta_4 = ab = xy$.

(2) Suppose that $k = q$ and $k = q - 1$ are satisfiable. We obtain that $\beta_q = ba = yx$ and $\beta_{q-1} = ab = xy$ if q is odd and $\beta_q = ab = xy$ and $\beta_{q-1} = ba = yx$ if q is even.

(3) $k = q + 1$. $F_k = F_{q+1} = \alpha_{q+1}\beta_{q+1} = F_qF_{q-1} = \alpha_q\beta_q\alpha_{q-1}\beta_{q-1}$. We obtain that $\beta_{q+1} = \beta_{q-1}$. Therefore, $\beta_{q+1} = \beta_{q-1} = ab = xy$ if q is odd ($q + 1$ is even) and $\beta_{q+1} = \beta_{q-1} = ba = yx$ if q is even ($q + 1$ is odd). $\square$

In fact, theorem 1 implies that β_{k+1} is the reversal of β_k ($\beta_{k+1} = \beta_k^r$) for $k \geq 3$. Therefore, we want to prove that α_k is a palindrome for $k \geq 3$.

Theorem 2. For $k \geq 3$, $F_k = \alpha_k \beta_k$ where β_k is the last two characters of F_k and α_k is a palindrome.

Proof. ($F_1 = x = a$, $F_2 = y = b$.)

(1) When $k = 3$, $F_k = F_3 = F_2F_1 = ba$. $\alpha_k = \alpha_3 = \varepsilon$ is satisfiable. When $k = 4$, $F_k = F_4 = F_3F_2 = bab$. $\alpha_k = \alpha_4 = b$ is satisfiable. When $k = 5$, $F_k = F_5 = F_4F_3 = babba$. $\alpha_k = \alpha_5 = bab$ is satisfiable.

(2) Suppose that $k = q$, $k = q - 1$ and $k = q - 2$ are satisfiable. We obtain that α_q , α_{q-1} and α_{q-2} are palindromes.

(3) $k = q + 1$. $F_k = F_{q+1} = \alpha_{q+1}\beta_{q+1} = F_qF_{q-1} = \alpha_q\beta_q\alpha_{q-1}\beta_{q-1}$. By theorem 1, $\beta_{q+1} = \beta_{q-1}$, we obtain $\alpha_{q+1} = \alpha_q\beta_q\alpha_{q-1}$. And we know that $\alpha_q\beta_q = F_q = F_{q-1}F_{q-2} = \alpha_{q-1}\beta_{q-1}\alpha_{q-2}\beta_{q-2}$. Therefore, $\alpha_{q+1} = \alpha_{q-1}\beta_{q-1}\alpha_{q-2}\beta_{q-2}\alpha_{q-1}$. We obtain that $\alpha_{q+1} = \alpha_{q+1}^r$ (because $\beta_{q-1} = \beta_{q-2}^r$). $\square$

By theorem 2, we know that there are recursive palindromes in Fibonacci string. Besides, Knuth showed an interesting and important theorem in [1].

Theorem 3. $c(F_k)$ denotes changing the two rightmost characters of F_k for $k \geq 3$. $c(F_k) = c(F_{k-1}F_{k-2}) = F_{k-2}F_{k-1}$.

Theorem 3 had been proved by induction in [1]. For example, $F_1 = a$ and $F_2 = b$. We obtain that $F_3 = ba$, $F_4 = bab$, $F_5 = babba$ and $F_6 = babbabab$. If we change the last two right most characters of F_6 ($c(F_6) = babbabba$), we obtain that $c(F_6) = F_4F_5$. Theorem 3 implies that F_k is a prefix and a suffix of $c(F_{k+1})$ for $k > 3$. Although $c(F_{k+1})$ is changing the two right most characters of F_{k+1} , we find that if a Fibonacci string F_{k+1} for $k > 3$ exists in a prefix of pattern and the last second position of F_{k+1} is i , we know that $failure[i] = |F_k| - 1$ where $|F_k|$ is the length of F_k . For above example, $F_6 = babbabab$, as show in Figure 2a and 2b, and F_6 is a prefix of pattern.

$$\begin{array}{r} 12345678 \\ c(F_6) = \text{babbabba} \\ F_5 = \text{babba} \end{array}$$

(a) F_5 and $c(F_6)$

$$\begin{array}{r} 12345678 \\ F_6 = \text{babbabab} \\ F_6 = \frac{\text{babbabab}}{F_5} \end{array}$$

(b) F_5 and F_6

Figure 2: The relation between F_5 and F_6

In Figure 2(b), we can observe that $i = 7$ is the last second position of F_6 . If we shift the last second position $j = 4$ of F_5 to i , we obtain that the longest prefix of $P(1, j - 1) = P(1, 4 - 1) = P(1, 3)$ is a suffix of $P(1, i - 1) = P(1, 7 - 1) = P(1, 6)$ and $p_i = p_7 = a \neq b = p_j = p_4$. Therefore, $failure[i] = failure[|F_6| - 1] = failure[7] = |F_5| - 1 = 5 - 1 = 4$. We call the last second position of a Fibonacci string F_k for $k \geq 5$ is critical point. For example, F_6 exists in the prefix of pattern. The critical points are 7 ($|F_6| - 1$) and 4 ($|F_5| - 1$). A critical point $i = 4$ means that F_5 exists in a prefix of pattern. We know that $F_1 = x$, $F_2 = y$, $F_3 = yx$ and $F_4 = yxy$. These strings almost exist in prefixes of pattern. We just want to find the Fibonacci strings F_k for $k \geq 5$. Therefore, critical points must equal to or large than $|F_5| - 1 = 5 - 1 = 4$.

After we know the above properties of Fibonacci string, we explain the delayed situation by using the KMP algorithm. When any critical point exists in a prefix of pattern (it means that Fibonacci strings exist in prefixes of pattern), it may cause the delayed situation by using the KMP algorithm. Such delayed situation must conform

Table 2: *failure* table of $P = babbababacb$

i	1	2	3	4	5	6	7	8	9	10	11
Pattern	b	a	b	b	a	b	a	b	a	c	b
<i>failure</i> [i]	0	1	0	2	1	0	4	0	4	1	0

to two conditions. First, when we compare the window with pattern, the first mismatch occurs in a critical point. Second, the mismatched character of the window is z that $z \neq x$ and $z \neq y$ (x and y are F_1 and F_2 respectively). We can review the example of Figure 1. The critical points are 4, 7 and 12. When we compare W with pattern by using the KMP algorithm, it satisfies the two conditions of delayed situation. Therefore, it always compares the location $j + 11$ of W .

If the beginning of a delayed situation is at the critical point $|F_k| - 1$, we use m' to record the length of F_k ($m' = |F_k|$). In Figure 1, we see that the beginning of a delayed situation is at $|F_7| - 1 = 13 - 1 = 12$. Therefore, $m' = |F_7| = 13$. In [1], it said that the number of comparisons of a delayed situation is bounded by a function of the approximate form $\log_\phi m'$, where ϕ is the golden ration ($\phi = (1+\sqrt{5})/2 \approx 1.618\dots$).

Although the number of alphabet of Fibonacci string is two, the first condition of delayed situation indicates that Fibonacci string only exists in prefixes of pattern. Hence, the delayed situation still occurs probably that the number of alphabet of pattern is large than two.

4 Improving the KMP algorithm

If we want to avoid the delayed situation, we must find critical points first. In fact, the *failure* table of the KMP algorithm had recorded the information. For example, $P = babbababacb$. A Fibonacci string F_6 is its prefix. And the *failure* table of P is shown in Table 2.

The largest critical point is $|F_6| - 1 = 7$. And $\text{failure}[|F_6| - 1] = \text{failure}[7] = 4$ is also a critical point. Therefore, we can use this concept to find all critical points from the smallest critical point 4 in pattern. We use a table, which is called critical points of Fibonacci string table (*CPF* table), to record critical points. And the size of *CPF* table is m . The initial value of $\text{CPF}[i]$ is 0 for $1 \leq i \leq m$. If the location i is a critical point and $P(1, i + 1)$ is a Fibonacci string, $\text{CPF}[i] = i$ and $\text{CPF}[i + 1] = -1$. Hence, we have Algorithm 1.

For above example, $P = babbababacb$. By using Algorithm 1, we obtain *CPF* table of pattern as

Algorithm 1: Construct *CPF* table by using *failure* table of the KMP algorithm.

Input : The *failure* table and pattern.

Output: *CPF* table of pattern.

Step 1 For i from 1 to m , $\text{CPF}[i] = 0$.

Step 2 $i = 4$ and $j = 2$. ($|F_5| - 1 = 5 - 1 = 4$ and $|F_4| - 1 = 3 - 1 = 2$.)

Step 3 If $i \geq m$, go to Step 6.

Step 4 If $\text{failure}[i] = j$ and $p_{i+1} = p_j$, $\text{CPF}[i] = i$ and $\text{CPF}[i + 1] = -1$. Otherwise, go to Step 6.

Step 5 If $i + j + 1 \geq m$, go to Step 6. Otherwise, $j = i$, $i = \text{failure}[i] + i + 1$ and go to Step 4.

Step 6 Return *CPF* table.

shown in Table 3. $\text{CPF}[4]$ and $\text{CPF}[7]$ are large than 0. It means that locaitons 4 and 7 are critical points of pattern. The values of $\text{CPF}[4 + 1] = \text{CPF}[5]$ and $\text{CPF}[7 + 1] = \text{CPF}[8]$ both are -1. It means that $P(1,5)$ and $P(1,8)$ are Fibonacci strings.

By using *CPF* table, we can easily to check whether a location is a critical point. Suppose that the first mismatched location is i in P . When $\text{CPF}[i] > 0$ and $\text{CPF}[i + 1] = -1$, i is a critical point and $P(1, i + 1)$ is a Fibonacci string. The second condition of the delayed situation is that mismatched character of W is z and $z \neq x \neq y$. We use a simple method to test whether z exists in $P(1, i + 1)$ or not.

If the mismatched character p_i is x and there exists a location i' that $\text{CPF}[i'] = -1$ ($i' \geq i$), $p_{\text{failure}[i]}$ must be y . If the mismatched character p_i is y and $\text{CPF}[i'] = -1$, $p_{\text{failure}[i]}$ must be x . So, we only need to check whether z is equal to $p_{\text{failure}[i]}$ or not. If $z \neq p_i = x$ and $z \neq p_{\text{failure}[i]} = y$, we know that z does not exist in $P(1, i + 1)$. Therefore, we can shift the window $\text{CPF}[i]$ steps.

For example, $W = babbabcbabc$ and $P = babbabababacb$. If the first location of W is j , we find the first mismatch between W and P is in loca-

Table 3: *CPF* table of $P = babbababac$

i	1	2	3	4	5	6	7	8	9	10	11
Pattern	b	a	b	b	a	b	a	b	a	c	b
$failure[i]$	0	1	0	2	1	0	4	0	4	1	0
$CPF[i]$	0	0	0	4	-1	0	7	-1	0	0	0

tion $j' + 6$. And the character of location $j' + 6$ in T will be compared with 4 times. But, if we use the improved method and utilize *CPF* table of pattern, we find that it conforms to the all conditions of delayed situation ($CPF[7] > 0$, $c \neq p_7 = a$ and $c \neq p_{failure[7]} = p_4 = b$). Hence, we can shift the window 7 steps directly. The delayed situation will not occur.

Besides, we also can improve the case that only first condition satisfies. When the mismatched location i is a critical point but $z = p_{failure[i]}$, we compare p_{i+1} with the character z' of location $i + 1$ in W . If $p_{i+1} \neq z'$, we can shift the window $failure[i + 1]$ steps. This method can avert that p_i compares with z again by using the original KMP algorithm.

By using the above improved method, if such delayed situations occur (two conditions are satisfiable), we can reduce the character comparison $\Sigma \log_\phi m'_g$ ($1 \leq g \leq h$. h is the total number of locations of delayed situation in text.) to $2h$ (the coefficient 2 is to check whether $x \neq z$ and $y \neq z$ or not).

The 24th Workshop on Combinatorial Mathematics and Computation Theory, pages 182-188, 2007.

- [3] Luca, A. A combinatorial property of the Fibonacci words. *Information Processing Letters*, 12(4):193-195, 1981.
- [4] Luca, A. A division property of the Fibonacci word. *Information Processing Letters*, 54(6):307-312, 1995.

5 Conclusion

In this paper, we only discuss the basic Fibonacci string. This Fibonacci string only contains two different alphabets. It seriously affects the KMP algorithm. Maybe the variations of Fibonacci string have other properties which will influence the performance of the KMP algorithm, too. Further, it is worth consideration that an exactly string matching algorithm will be concerned if some special properties exist in pattern.

References

- [1] Knuth, D. E., Morris J. H. and Pratt V. R. Fast pattern matching in strings. *SIAM Journal Computing*, 6(2):323-349, 1977.
- [2] Lin, J. S., Juan, L. C. and R. C. T. Lee. The generation of words with special properties. *In*